ctwc/checked_attributes/macro.rb

```
class Class
  def attr_checked(attribute, &validation)
    define_method "#{attribute}=" do |value|
      raise 'Invalid attribute' unless validation.call(value)
      instance_variable_set("@#{attribute}", value)
    end
    define_method attribute do
      instance_variable_get "@#{attribute}"
    end
  end
end
```

This code doesn't even need to call to class_eval, because when the method executes, the class is already taking the role of self.

"That's great," Bill says. "One more step, and we'll be done." For this last step, however, you need to learn about a feature that we haven't talked about yet: Hook Methods.

Hook Methods

Where you get one of Bill's thorough lessons in advanced coding.

The object model is an eventful place. Lots of things happen there as your code runs: classes are inherited, modules are mixed into classes, and methods are defined, undefined, and removed. Imagine if you could "catch" these events like you catch GUI mouse-click events. You'd be able to execute code whenever a class is inherited or whenever a class gains a new method.

Well, it turns out you can do all these things. This program prints a notification on the screen when a class inherits from String:

ctwc/hooks.rb

```
class String
  def self.inherited(subclass)
    puts "#{self} was inherited by #{subclass}"
  end
end

class MyString < String; end
```

```
String was inherited by MyString
```

The inherited method is an instance method of Class, and Ruby calls it when a class is inherited. By default, Class#inherited does nothing, but you can override it with your own code as in the earlier example. A method such as Class#inherited is called a *Hook Method* because you can use it to hook into a particular event.

Spell: Hook Method